

Bounded Concurrency

A design essay (not a probe finding) explaining how the chat app limits simultaneous in-flight calls to `/weather` and `/research`, what that does and does not protect against, and the trade-off vs. proactive pacing. References `agent.py`, `elyos_client.py`, and `parsers/research.py`.

TL;DR

The chat app uses an `asyncio.Semaphore(1)` for **both** `/weather` and `/research`, meaning at most one call to each endpoint is in flight at a time per LLM turn. Cross-endpoint calls (one weather + one research) can still overlap. This is **bounded concurrency, not pacing**: the semaphores limit in-flight count, they do not track or reserve server rate-limit slots. When a throttle does occur, the client handles it reactively via `elyos_client.py`'s retry loop.

Note on current values: With both endpoints at `max_concurrent: 1`, the per-endpoint semaphore infrastructure in `agent.py` is mostly inert in practice — only cross-endpoint overlap remains. The infrastructure is kept because the trade-off discussion below applies whenever `max_concurrent > 1` might be considered.

Current Design Summary

CAPABILITY	PRESENT?	WHERE
Bounded concurrency	Yes	<code>agent.py</code> — per-endpoint <code>asyncio.Semaphore</code>
Proactive pacing / local budget tracking	No	—
Reactive throttle retry	Yes	<code>elyos_client.py</code> — sleeps <code>retry_after + 1</code> , up to 2 retries

CAPABILITY	PRESENT?	WHERE
Empty <code>{}</code> timeout parsing	Yes	<code>parsers/research.py</code> — converts to <code>ResearchResult(kind="timeout")</code>
Cancellation warning	Yes	<code>cli_chat.py</code> — warns the interrupted call may still count against the rate limit

Bounded Concurrency Is Not Pacing

These are two distinct strategies. The chat app implements only the first.

	BOUNDED CONCURRENCY	PROACTIVE PACING
What it controls	Maximum simultaneous in-flight HTTP calls per endpoint	Timing of calls against a local mirror of the server's sliding rate-limit window
What it knows	How many calls are currently running	How many rate-limit slots are available right now
Prevents throttle?	Reduces burst pressure, but cannot prevent throttle from budget already consumed by other endpoints	Can delay a call until a slot opens, avoiding most throttles proactively
Cross-endpoint awareness	No — each endpoint has its own semaphore	Yes — a single shared budget covers all endpoints

Key distinction: Bounded concurrency limits how many calls run *simultaneously*. Pacing limits how many calls run *within a time window*. Both `/weather` and `/research` share the same server-side rate-limit bucket (~5 requests per 30 s), but the semaphores in `agent.py` are independent per endpoint.

Why Both Endpoints Use `max_concurrent: 1`

`/research` is the stronger case: it is slow (~3–8 s on normal calls, ~15 s on the server-side timeout that returns an empty `{}` body), and probe runs showed concurrent research fan-out performing poorly — under 5 parallel calls, some succeeded, one returned `{}`, and two gave up after exhausting throttle retries.

`/weather` is fast (~200 ms per call) and previously ran with `max_concurrent: 4`. It was rolled back to `1` after noticing the same shared-budget logic applies: bursting four weather calls at once eats four of the ~5 slots in the 30 s window in well under a second, with no real wall-clock win because the LLM rarely emits more than 2–3 tool calls per round. Serialising both endpoints keeps behaviour predictable and avoids the temptation to interpret "bounded" as "rate-aware."

Serialising calls within an endpoint avoids several failure modes:

- **Competing retries.** If two parallel research calls both hit a throttle, they both sleep and retry simultaneously, doubling the burst pressure when the window reopens.
- **Retry storms from fan-out.** With 5 parallel calls into a partially full budget, multiple calls may throttle at once, and their staggered retries can cascade.
- **Wasted timeout slots.** A concurrent research call that times out after 15 s still consumed a rate-limit slot, reducing budget for the calls that did return useful data.

Trade-off: Serialisation makes multi-research turns slower in wall-clock time (e.g., 3 calls × 5 s = ~15 s sequentially vs. ~5 s concurrent). For a CLI chat app, predictable behaviour is more valuable than raw speed.

What Bounded Concurrency Protects Against

THREAT	PROTECTED?	MECHANISM
Multiple research calls from one LLM turn all firing at once	Yes	Semaphore(1) serialises them
Competing retries from parallel research calls	Yes	Only one research call is active, so only one retry loop runs
Overloading a slow endpoint in a single turn	Yes	Calls queue behind the semaphore

What It Does Not Protect Against

THREAT	WHY NOT	WHAT HANDLES IT INSTEAD
Shared bucket exhaustion from prior <code>/weather</code> calls	Research semaphore has no visibility into weather usage	Reactive throttle retry in <code>elyos_client.py</code>
Server returning HTTP 200 throttle envelope	Bounded concurrency does not inspect response bodies	Throttle detection + retry loop in <code>elyos_client.py</code>
Server-side <code>{}</code> timeout (~15 s)	Bounded concurrency does not know about response content	<code>parsers/research.py</code> converts to <code>kind="timeout"</code>
Cancelled calls consuming a rate-limit slot	Bounded concurrency releases the semaphore on cancel, but the server already counted the request	Warning in <code>cli_chat.py</code> ; reactive retry on next call
Stale cached research results (~2-year-old data)	Not a concurrency concern	<code>parsers/research.py</code> detects <code>cached: true</code> and humanises the age
Topic truncation at 50 characters	Not a concurrency concern	<code>parsers/research.py</code> detects <code>truncated: true</code>

Scenarios

SCENARIO A

LLM emits 3 research tool calls in one turn

The LLM returns a single assistant message containing 3 `research_topic` tool calls. `agent.py` gathers them with per-endpoint semaphores.

WITHOUT BOUNDED CONCURRENCY

```
t=0s call_1, call_2, call_3 all fire simultaneously
t=5s call_1 returns (4.8s)
t=6s call_2 returns (5.6s)
t=7s call_3 returns (6.2s)
```

3 API requests, ~7 s wall-clock. Fast if all succeed. But if the budget is tight, three simultaneous calls triple the chance of a throttle cascade.

WITH RESEARCH MAX_CONCURRENT = 1

```
t=0s call_1 fires (call_2, call_3 queued behind semaphore)
t=5s call_1 returns → call_2 fires
t=11s call_2 returns → call_3 fires
t=17s call_3 returns
```

3 API requests, ~17 s wall-clock. Slower, but each call completes before the next starts. No concurrent retry contention.

SCENARIO B

LLM emits 5 research tool calls — resembling the probe stress case

The LLM returns 5 `research_topic` calls. The shared API budget has 3 of 5 slots remaining.

WITHOUT BOUNDED CONCURRENCY

```
t=0s all 5 fire simultaneously
t=0s call_1, call_2, call_3 get server slots
t=0s call_4, call_5 throttled → sleep ~31s each
t=5s call_1, call_2 return
t=8s call_3 returns
t=31s call_4 retry → succeeds
t=31s call_5 retry → succeeds
```

7 API requests, ~36 s wall-clock. Two wasted requests on the initial throttles. Both retries fire simultaneously, recreating burst pressure.

WITH RESEARCH MAX_CONCURRENT = 1

```
t=0s call_1 fires
t=5s call_1 returns → call_2 fires
t=11s call_2 returns → call_3 fires (last slot)
t=17s call_3 returns → call_4 fires
t=17s call_4 throttled (budget exhausted) → sleep ~31s
t=48s call_4 retry → succeeds → call_5 fires
t=54s call_5 returns
```

6 API requests, ~54 s wall-clock. Slower total time, but only one throttle-and-retry cycle instead of two concurrent retries. The first 3 calls succeed cleanly because they never compete with each other

for server slots.

SCENARIO C

Prior `/weather` calls consumed the shared budget

The LLM's previous tool round made 4 weather calls, consuming 4 of 5 slots. It now emits 1 research call and 1 weather call.

```
prior turn: 4 × /weather → all succeed, 1 slot left in the 30s window
t=0s research_topic fires, weather fires (different semaphores)
t=0s one gets the last slot; the other is throttled
t=~31s throttled call retries → succeeds
```

Bounded concurrency cannot prevent this. The research semaphore has no visibility into weather's budget consumption. The throttle is handled reactively by `elyos_client.py`'s retry loop.

Limitation: Because per-endpoint semaphores are independent, and the server enforces a *shared* rate-limit bucket across both endpoints, bounded concurrency alone cannot prevent cross-endpoint budget exhaustion. Only proactive pacing with a single shared budget mirror could avoid this.

SCENARIO D

User cancels a research call mid-flight

The user interrupts at ~2 s into a ~6 s research call. Two transports support this:

- **CLI:** the user presses Ctrl+C; `cli_chat.py` catches SIGINT and cancels the turn task.
- **Web UI:** the user presses the cancel button; the frontend sends `{"type":"cancel"}` over the WebSocket and `ws_server.py` cancels the turn task.

```
t=0s research call fires
t=2s user cancels → asyncio.CancelledError
the turn task is cancelled
semaphore is released (async with exits)
but the server already received and counted the request
```

The rate-limit slot is consumed regardless of cancellation. `cli_chat.py` warns: "The interrupted API call may still count against the rate limit." The web UI marks the partial response as interrupted and emits a `cancelled` event followed by `done`. If the user's next message triggers another research call and the budget is tight, `elyos_client.py`'s throttle retry handles the reactive recovery.

Engineering Trade-off

Bounded concurrency is intentionally simpler than a full pacing layer:

- **Smaller codebase.** The semaphore logic is 6 lines in `agent.py`. A proactive pacer would add a shared timestamp deque, a lock, and budget-awareness across endpoints — roughly 40–60 additional lines.
- **Easier to reason about.** "At most N research calls run at once" is simpler to explain and debug than "calls are delayed until a local sliding-window mirror says a slot is available."
- **Accepts occasional reactive throttles.** When the budget is exhausted, `elyos_client.py`'s retry loop handles it. The cost is one wasted API request plus ~31 s of sleep per throttle — acceptable for a CLI chat with low concurrency.
- **Honest about what it does not do.** The system prompt and parsers surface rate-limit behaviour to the LLM and the user. No component claims to prevent all throttles.

Alignment: This approach keeps the take-home code smaller and easier to review. It trades a small number of reactive throttles for significantly less implementation complexity.

Do Not Claim

For accuracy, the following statements should **not** appear in documentation or system prompts for this codebase:

- The app prevents all throttles.
- The app tracks the 5-per-30 s budget locally.
- The app reserves rate-limit slots before making calls.

- Cancellation frees a server-side rate-limit slot.
- Retries use exponential backoff or jitter. *(They use the server's `retry_after_seconds` value + 1s fixed offset.)*
- The semaphore prevents throttles from the shared cross-endpoint budget.

Reactive Resilience Stack

Although the app does not proactively pace, it has a layered reactive defence:

LAYER	WHAT IT HANDLES	CONFIG
Bounded concurrency	Limits simultaneous in-flight calls per endpoint	<code>max_concurrent: 1</code> (both endpoints)
Throttle retry	Detects <code>{"status": "throttled"}</code> in 200 body, sleeps <code>retry_after + 1</code>	<code>max_throttle_retries: 2</code>
Timeout retry	Retries on <code>httpx.TimeoutException</code> (client-side timeout)	<code>timeout_s: 20</code> , <code>max_timeout_retries: 1</code>
Empty body parsing	Converts <code>{}</code> (server-side timeout) to structured <code>kind="timeout"</code>	—
Cancellation handling	Catches <code>CancelledError</code> , warns user about slot consumption	—

Code references: `agent.py` (semaphore creation and `_bounded_execute`), `elyos_client.py` (throttle retry and timeout retry), `parsers/research.py` (empty-body / cache detection), `cli_chat.py` (SIGINT cancellation warning), `interfaces/ws_server.py` (WebSocket cancel protocol), `config.yaml` (endpoint configuration).